

UNAM

Universidad Nacional Autónoma de México

Facultad de Ciencias

Modelado y Programación

Proyecto 02

Pong Evolved

Equipo Polimórfo

Juárez Larrondo Omar Alejandro
(322245244)

Soto Mendoza Ismael de Jesús
(322210422)

Vega Navas Saúl (322088267)

Identificación del Proyecto

Este documento corresponde al **Proyecto 02** de la materia *Modelado y Programación* de la Facultad de Ciencias, UNAM. El objetivo principal es diseñar e implementar un sistema integral de videojuego basado en el clásico Pong, extendido con funcionalidades modernas que incluyen múltiples modos de juego, sistema de items dinámicos, inteligencia artificial adaptativa, personalización de elementos, gestión de usuarios con sistema de logros y persistencia completa de datos. La solución propuesta integra doce patrones de diseño fundamentales que resuelven los desafíos arquitectónicos del problema mediante estructuras flexibles, extensibles y mantenibles que adhieren a principios SOLID.

1.1. Introducción

El juego Pong representa uno de los primeros videojuegos comercialmente exitosos de la historia, introducido por Atari en 1972 como simulación electrónica del tenis de mesa donde dos jugadores controlan paletas verticales para golpear una pelota evitando que atravesase su línea de defensa. A pesar de su impacto histórico y simplicidad conceptual que lo convierte en proyecto educativo ideal para comprender fundamentos de desarrollo de videojuegos, el Pong clásico enfrenta obsolescencia en el mercado contemporáneo dominado por productos con gráficos avanzados, mecánicas complejas y sistemas de progresión que mantienen engagement prolongado del jugador. La demanda actual de entretenimiento digital requiere experiencias que proporcionen variedad mediante múltiples modos de juego, personalización que permita expresión individual del jugador, y sistemas de recompensas que incentiven dedicación continua mediante logros desbloqueables y reconocimiento de habilidad.

El sistema desarrollado revitaliza el concepto clásico incorporando características modernas que expanden significativamente la jugabilidad sin comprometer la accesibilidad inmediata que caracteriza al Pong original. La implementación incluye cuatro modos de juego distintos que proporcionan experiencias diferenciadas: el modo clásico tradicional uno contra uno que preserva la experiencia original, el modo Atari Breakout inspirado en el juego de destrucción de bloques donde un jugador controla paleta inferior eliminando formaciones de bloques mediante impactos de pelota, el modo por obtención de puntos que establece victoria al alcanzar umbral predefinido incentivando juego agresivo, y el modo editor de mapas que permite diseño personalizado de niveles con colocación arbitraria de bloques creando desafíos únicos compartibles con otros jugadores.

La experiencia de juego se enriquece mediante sistema de items temporales que modifican dinámicamente comportamiento de elementos del juego durante duraciones limitadas, creando situaciones tácticas impredecibles que recompensan adaptabilidad del jugador. Los items disponibles incluyen espinas que dañan al oponente al contacto con su paleta, incremento temporal de velocidad de pelota que dificulta reacción defensiva, cajas aleatorias que otorgan efectos impredecibles, múltiples bolas simultáneas que saturan capacidad defensiva del oponente, redimensionamiento de paleta enemiga que restringe cobertura vertical, y neblina que oculta parcialmente la pelota dificultando seguimiento visual. Estos elementos transforman cada partida en experiencia única donde dominio de mecánicas básicas debe complementarse con gestión táctica de recursos temporales y adaptación rápida a condiciones variables del campo de juego.

El sistema incorpora inteligencia artificial con tres niveles de dificultad que proporcionan desafío apropiado para jugadores de diferentes habilidades, desde principiantes que requieren oponente predecible para aprender mecánicas básicas, hasta jugadores experimentados que demandan comportamiento sofisticado con predicción de trayectorias y respuesta prácticamente infalible que simula competencia humana de alto nivel. La personalización visual permite modificar colores de paletas individuales creando identidad visual distintiva, mientras el sistema de usuarios con autenticación persistente registra progresión individual mediante puntajes altos por modo de juego y logros desbloqueables que reconocen hitos específicos como victoria sin recibir punto, elección de dificultad máxima o creación de niveles personalizados distribuidos exitosamente.

La arquitectura del sistema se fundamenta en patrones de diseño que organizan responsabilidades mediante separación clara de lógica de negocio, presentación y control, garantizando que modificaciones futuras como adición de nuevos modos, items o estrategias de inteligencia artificial puedan implementarse mediante extensión sin modificación de código existente. Esta flexibilidad arquitectónica combinada con

persistencia completa de configuraciones, progresión de usuarios y niveles personalizados proporciona experiencia moderna y profesional que transforma el concepto clásico de Pong en producto competitivo capaz de atraer audiencia contemporánea mientras preserva la accesibilidad inmediata que caracteriza al diseño original.

Problemática

El videojuego Pong, a pesar de representar un hito histórico fundamental en la industria del entretenimiento electrónico y mantener relevancia pedagógica como introducción accesible a conceptos de desarrollo de videojuegos, enfrenta obsolescencia crítica en el mercado contemporáneo debido a limitaciones estructurales que restringen significativamente su capacidad de competir con productos modernos. La experiencia de juego tradicional ofrece únicamente modalidad uno contra uno con mecánicas estáticas donde paletas se desplazan verticalmente para interceptar pelota que rebota predeciblemente, agotando rápidamente el interés del jugador ante ausencia de variedad, progresión o elementos sorpresa que incentiven dedicación prolongada. Los consumidores actuales de entretenimiento digital demandan experiencias que proporcionen múltiples modos de juego con reglas diferenciadas creando desafíos distintos que requieren desarrollo de habilidades especializadas, sistemas de personalización que permitan expresión de identidad individual mediante modificación de apariencia visual y configuración de elementos del juego, progresión cuantificable mediante puntajes persistentes y logros desbloqueables que reconozcan hitos específicos incentivando dedicación continua, y oponentes controlados por inteligencia artificial con múltiples niveles de dificultad que proporcionen desafío apropiado según habilidad del jugador sin requerir segundo participante humano.

La revitalización exitosa del concepto Pong requiere diseño arquitectónico que soporte extensibilidad sistemática mediante incorporación de nuevos modos de juego sin modificación de lógica central, integración de items temporales que modifiquen dinámicamente comportamiento de elementos creando situaciones tácticas impredecibles, y personalización de niveles mediante editor que permita creación de formaciones arbitrarias de bloques compartibles con otros jugadores. Estos requisitos funcionales deben implementarse mediante estructuras de software que minimicen acoplamiento entre componentes para facilitar evolución independiente de subsistemas, maximicen reusabilidad de código mediante abstracciones que permitan tratamiento polimórfico de elementos heterogéneos, y garanticen mantenibilidad a largo plazo mediante organización clara de responsabilidades que simplifique comprensión del sistema y localización de defectos.

Los desafíos técnicos fundamentales incluyen gestión de colisiones entre objetos móviles con formas geométricas diversas requiriendo detección precisa de intersecciones y cálculo de ángulos de rebote que preserven realismo físico, coordinación de múltiples elementos simultáneos como pelotas, paletas, bloques e items activos que interaccionan mediante reglas complejas dependientes del contexto, persistencia de estados complejos que incluyen configuraciones de usuarios, progresión individual, niveles personalizados y sesiones de juego pausadas que deben restaurarse preservando integridad de relaciones entre objetos, inteligencia artificial que calcule movimientos óptimos mediante análisis predictivo de trayectorias considerando velocidad y dirección de pelota junto con posición actual de paleta, sistema de observación que notifique eventos relevantes a múltiples subsistemas interesados como interfaz gráfica, reproductor de audio y gestor de logros sin acoplamiento directo entre emisor y receptores, y arquitectura de items temporales que modifiquen comportamiento de objetos host durante duraciones limitadas aplicando transformaciones reversibles que restauren estado original al expirar efecto.

La solución propuesta debe evitar rigidez que dificulte modificaciones mediante dependencias explícitas a clases concretas, fragilidad donde cambios localizados provoquen fallas en módulos aparentemente no relacionados, inmovilidad que impida reuso de componentes en contextos distintos debido a dependencias innecesarias con subsistemas externos, y viscosidad donde modificaciones correctas requieran esfuerzo desproporcionado incentivando soluciones subóptimas que degraden calidad arquitectónica. El sistema debe implementar separación estricta entre lógica de negocio independiente de presentación, coordinación de flujo mediante controladores que medien interacciones sin contener lógica de dominio, y renderizado visual mediante vistas especializadas que traduzcan estado abstracto del modelo a representaciones gráficas concretas. Esta organización mediante patrón arquitectónico MVC proporciona base estructural sobre la cual patrones adicionales resuelven desafíos específicos de extensibilidad, compo-

sición, comportamiento dinámico y gestión de dependencias, resultando en sistema profesional capaz de evolucionar sosteniblemente respondiendo a requisitos emergentes sin acumular deuda técnica que eventualmente comprometa viabilidad del proyecto.

Propuesta de Solución

El sistema desarrollado implementa arquitectura modular fundamentada en doce patrones de diseño que colaboran coordinadamente para proporcionar solución integral a los desafíos identificados en la problemática, organizando responsabilidades mediante estructuras que maximizan flexibilidad, extensibilidad y mantenibilidad del código resultante. La arquitectura general se fundamenta en el patrón MVC que separa estrictamente tres capas fundamentales: el modelo que encapsula estado del juego incluyendo posiciones de todos los objetos, velocidades, puntajes, items activos y lógica de actualización que procesa física de colisiones y transiciones de estado, la vista que renderiza representación gráfica del modelo mediante JavaFX con componentes visuales como lienzo para dibujo de elementos del juego, etiquetas para visualización de puntajes y paneles para menús de configuración, y el controlador que coordina interacciones procesando eventos de entrada del usuario, delegando operaciones al modelo apropiado y actualizando vista para reflejar cambios en el estado observable.

La gestión de objetos del juego aprovecha el patrón Composite que establece jerarquía parte-todo donde elementos atómicos como pelotas individuales, paletas y bloques se tratan uniformemente con composiciones que agrupan múltiples objetos bajo interfaz común, permitiendo operaciones como actualización y dibujo mediante invocaciones polimórficas que procesan recursivamente estructuras complejas sin requerir distinciones explícitas entre hojas y nodos compuestos. Esta abstracción simplifica significativamente lógica del modelo que manipula colecciones heterogéneas de objetos mediante operaciones genéricas aplicables transparentemente a elementos individuales o agrupaciones arbitrarias.

El sistema de colisiones implementa patrón Strategy mediante encapsulación de algoritmos especializados de detección y resolución en clases intercambiables registradas dinámicamente en gestor que selecciona estrategia apropiada según tipos de objetos involucrados, permitiendo comportamientos diferenciados como rebote de pelota contra paleta con cálculo de ángulo basado en punto de impacto, destrucción de bloque con generación probabilística de ítem, y rebote simple contra paredes. Esta separación de algoritmos en estrategias independientes facilita extensión con nuevos tipos de colisiones sin modificar lógica central del gestor, adhiriendo al principio abierto-cerrado que establece que módulos deben ser abiertos para extensión pero cerrados para modificación.

Paralelamente, el movimiento de paletas controladas por inteligencia artificial utiliza patrón Strategy adicional donde diferentes niveles de dificultad se implementan como estrategias intercambiables que calculan movimiento óptimo mediante algoritmos de complejidad variable, desde seguimiento básico de posición vertical de pelota en dificultad fácil, hasta predicción sofisticada de trayectoria futura considerando velocidad y rebotes múltiples en dificultad alta. El servicio de inteligencia artificial actúa como contexto que mantiene referencia a estrategia actual y delega cálculo de movimientos, permitiendo cambio dinámico de dificultad durante ejecución sin reinicializar estado del juego.

La creación de estrategias de inteligencia artificial se gestiona mediante patrón Factory que encapsula lógica de instanciación en factoría especializada que retorna implementación apropiada según nivel de dificultad solicitado, ocultando detalles de construcción y dependencias de cada estrategia concreta al código cliente que simplemente especifica dificultad deseada recibiendo objeto funcionalmente completo. Similarmente, la generación de niveles predefinidos utiliza factoría adicional que crea instancias de nivel con configuraciones optimizadas para diferentes grados de desafío, proporcionando catálogo de mapas probados que jugadores pueden seleccionar sin requerir diseño manual mediante editor.

Los modos de juego implementan patrón Template Method mediante clase abstracta que define esqueleto del algoritmo de ejecución con métodos gancho que subclases concretas especializan para proporcionar comportamiento específico, permitiendo reutilización de lógica común como inicialización de modelo y bucle principal de actualización mientras delegan verificación de condiciones de victoria y derrota a implementaciones específicas de cada modo. Esta estructura facilita incorporación de nuevos modos mediante extensión de plantilla abstracta sin duplicación de código compartido entre modalidades

existentes.

La construcción de niveles personalizados mediante editor utiliza patrón Builder que proporciona interfaz fluida para especificación incremental de propiedades como nombre, dificultad y colección de bloques con tipos y posiciones arbitrarias, separando algoritmo de construcción complejo de representación final del producto. El método construir retorna instancia de nivel completamente inicializada y validada, encapsulando verificaciones de consistencia que garantizan integridad de configuraciones creadas por usuarios.

La personalización de paletas implementa patrón Prototype mediante registro de configuraciones pre-definidas que sirven como prototipos clonables, permitiendo creación rápida de paletas personalizadas mediante copia de configuración base seguida de modificaciones incrementales que ajustan propiedades específicas sin recrear objeto desde cero. Esta aproximación optimiza uso de memoria y simplifica gestión de variantes cuando diferencias entre configuraciones son mínimas.

La gestión de puntajes, configuración global y prototipos de paletas utiliza patrón Singleton que garantiza existencia de instancia única compartida globalmente, proporcionando punto de acceso centralizado que coordina estado común sin proliferación de instancias redundantes que complicarían sincronización. Los gestores singleton implementan inicialización perezosa que pospone creación hasta primera solicitud, optimizando uso de recursos cuando funcionalidad no es requerida durante sesión particular.

El sistema de usuarios implementa patrón Decorator que agrega dinámicamente capacidad de gestión de logros a usuarios sin modificar clase base, permitiendo composición recursiva donde decoradores encadenan funcionalidad adicional manteniendo compatibilidad con interfaz original. Esta estructura facilita extensiones futuras como decoradores para estadísticas detalladas o integración con redes sociales mediante wrapping adicional sin impactar implementaciones existentes.

La notificación de eventos del juego aprovecha patrón Observer donde modelo actúa como sujeto observable que notifica automáticamente cambios relevantes a múltiples observadores registrados incluyendo interfaz de usuario que actualiza visualización de puntajes, reproductor de audio que ejecuta efectos sonoros contextuales, y gestor de logros que verifica cumplimiento de condiciones de desbloqueo. Esta arquitectura elimina acoplamiento directo entre modelo y subsistemas interesados permitiendo agregar observadores adicionales sin modificar lógica del sujeto observable.

La persistencia de datos se gestiona mediante servicios especializados que serializan estado completo del juego, configuraciones de usuarios, niveles personalizados y puntajes altos en formatos estructurados almacenados en sistema de archivos, permitiendo restauración completa de sesiones y continuidad de progresión entre ejecuciones. La implementación utiliza biblioteca Gson para serialización automática de objetos Java a formato JSON legible que facilita depuración e integración con herramientas externas.

La tecnología empleada incluye JavaFX como framework de interfaz gráfica que proporciona componentes visuales avanzados con renderizado acelerado por hardware mediante Canvas que dibuja primitivas geométricas representando elementos del juego, sistema de eventos que captura interacciones del usuario mediante listeners registrados en controlador, y layouts responsivos que organizan componentes adaptándose a dimensiones variables de ventana. La detección de colisiones aprovecha Rectangle2D que representa cajas delimitadoras de objetos mediante coordenadas y dimensiones, proporcionando método intersects que verifica solapamiento entre rectángulos indicando potencial colisión. El bucle principal de juego implementa AnimationTimer que invoca callback de actualización sincronizado con frecuencia de refresco de pantalla, garantizando animaciones fluidas mediante cálculo de delta temporal entre frames que compensa variaciones en velocidad de procesamiento.

Esta combinación de patrones arquitectónicos y tecnologías modernas resulta en sistema profesional que satisface requisitos funcionales identificados en la problemática mientras mantiene estructuras de código que facilitan comprensión, extensión y mantenimiento a largo plazo, proporcionando base sólida para evolución sostenible del producto sin acumulación de deuda técnica que eventualmente comprometa viabilidad del proyecto.

Análisis de Implementación de Patrones

El sistema integra doce patrones de diseño fundamentales que trabajan coordinadamente para proporcionar arquitectura flexible, extensible y mantenible que resuelve eficazmente los requisitos del problema planteado mientras adhiere a principios SOLID que minimizan acoplamiento y maximizan cohesión entre componentes.

4.1. Patrón MVC (Model-View-Controller)

El patrón MVC constituye la arquitectura fundamental del sistema, organizando responsabilidades mediante separación estricta entre tres capas que colaboran sin acoplamiento directo. Esta elección se fundamenta en la necesidad de aislar lógica de negocio independiente de detalles de presentación, permitiendo que modelo evolucione sin impactar vista y facilitando implementación de interfaces alternativas que visualicen mismo estado subyacente mediante representaciones diferenciadas.

El modelo se materializa en la clase `ModeloJuego` que encapsula estado completo del juego incluyendo pelota con posición y velocidad bidimensional, paletas controladas por jugador e inteligencia artificial con estrategias de movimiento intercambiables, bloques destructibles e indestructibles organizados en formaciones arbitrarias, items activos que modifican temporalmente comportamiento de elementos, gestores de colisiones que procesan interacciones entre objetos, y puntajes acumulados por cada jugador durante sesión actual. El modelo proporciona métodos de actualización que calculan nuevo estado basado en tiempo transcurrido desde frame anterior, procesando movimiento de elementos según velocidades actuales, verificando colisiones mediante delegación a estrategias especializadas, actualizando duraciones de items temporales desactivando automáticamente efectos expirados, e incrementando puntajes cuando pelota atraviesa línea de defensa enemiga. Adicionalmente, el modelo expone operaciones de consulta que retornan información observable sin modificar estado interno, permitiendo que vista renderice representación actualizada sin capacidad de corrupción accidental mediante invocaciones inadecuadas.

La vista se implementa mediante la clase `VistaJuego` que gestiona interfaz gráfica JavaFX con componente `Canvas` donde renderiza representación visual de todos los objetos del modelo mediante primitivas geométricas como rectángulos para paletas y bloques, círculos para pelota, y texto para visualización de puntajes. La vista mantiene referencia al contexto gráfico del canvas que proporciona métodos de dibujo como `fillRect`, `fillOval` y `strokeText` que traducen coordenadas abstractas del modelo a píxeles concretos en pantalla. El método `renderizar` invoca secuencialmente operaciones de dibujo para cada elemento visible, limpiando previamente el lienzo para prevenir acumulación de frames anteriores que producirían efecto de estela indeseado. Adicionalmente, la vista gestiona etiquetas de texto que muestran puntajes actualizados dinámicamente mediante método `actualizarPuntaje` invocado por controlador después de modificaciones relevantes en modelo, y paneles de menú para configuración de opciones como dificultad de inteligencia artificial, selección de modo de juego y personalización de colores de paleta.

El controlador se materializa en la clase `ControladorJuego` que coordina interacciones entre modelo y vista sin contener lógica de dominio, actuando como mediador que traduce eventos de interfaz a operaciones de modelo y desencadena actualizaciones de vista para reflejar cambios en estado observable. El controlador mantiene referencias a instancias de modelo y vista junto con adaptador de entrada que captura eventos de teclado registrando teclas presionadas y liberadas en mapa que representa estado actual del input. El método `iniciar` configura bucle principal de juego mediante `AnimationTimer` que invoca `callback handle` en cada frame, calculando tiempo transcurrido desde actualización anterior y delegando procesamiento al modelo que avanza simulación, seguido de invocación a vista que renderiza estado resultante. Los métodos `pausar` y `reanudar` gestionan ciclo de vida del bucle de juego permitiendo suspensión temporal de simulación sin perder estado, facilitando implementación de menú de pausa que presenta opciones al jugador sin requerir reinicio completo de partida.

La integración del patrón MVC en el sistema permite desarrollar modelo mediante pruebas unitarias que verifican lógica de actualización sin requerir interfaz gráfica, facilitando detección temprana de defectos mediante validación automatizada. Paralelamente, la vista puede diseñarse y refinarse independientemente ajustando colores, tamaños y animaciones sin riesgo de corromper lógica de negocio. El controlador encapsula coordinación simplificando comprensión del flujo de control mediante centralización de interacciones entre capas que de otro modo requerirían referencias cruzadas complejas entre múltiples clases.

Esta separación de responsabilidades maximiza mantenibilidad permitiendo que equipos especializados trabajen concurrentemente en capas distintas minimizando conflictos de integración.

4.2. Patrón Composite

El patrón Composite fue implementado para gestionar objetos del juego que pueden ser tratados tanto individualmente como en agrupaciones mediante jerarquías parte-todo que unifican componentes atómicos y compuestos bajo interfaz común. Esta elección responde al requisito de procesar colecciones heterogéneas de elementos mediante operaciones uniformes como actualización de posiciones, dibujo de representaciones gráficas y obtención de cajas delimitadoras para detección de colisiones, sin requerir distinciones explícitas basadas en tipo concreto de objeto.

La interfaz `ComponenteJuego` establece contrato abstracto definiendo operaciones fundamentales que todos los objetos del juego deben implementar, incluyendo `actualizar` que procesa lógica de comportamiento avanzando estado interno según tiempo transcurrido, `dibujar` que renderiza representación visual en contexto gráfico proporcionado, `obtenerLimites` que retorna `Rectangle2D` representando caja delimitadora utilizada por sistema de colisiones, y métodos de consulta `obtenerX` y `obtenerY` que retornan coordenadas de posición actual. Esta abstracción permite que código cliente manipule cualquier objeto del juego mediante referencias polimórficas invocando operaciones genéricas sin conocimiento del tipo específico subyacente.

La clase abstracta `ObjetoJuego` implementa `ComponenteJuego` proporcionando implementación base con atributos `x` y `y` que representan posición bidimensional en sistema de coordenadas del juego, junto con implementaciones por defecto de getters que retornan valores almacenados. Las subclasses concretas extienden `ObjetoJuego` especializando comportamiento mediante implementación de métodos abstractos `actualización` y `dibujar` según semántica específica de cada tipo de elemento.

La clase `Pelota` representa componente hoja con atributos adicionales `velocidadX` y `velocidadY` que determinan dirección y magnitud de movimiento, `radio` que define tamaño del círculo dibujado, `rapidez` que escala magnitud de vector de velocidad, y `color` que especifica apariencia visual. El método `actualizar` incrementa posición mediante suma de velocidades multiplicadas por delta temporal, implementando integración numérica simple que aproxima movimiento continuo. El método `dibujar` invoca `fillOval` en contexto gráfico renderizando círculo centrado en posición actual con diámetro determinado por radio. La clase proporciona operaciones especializadas como `establecerVelocidad` que modifica dirección preservando magnitud, `incrementarRapidez` que aplica factor multiplicativo a velocidad simulando aceleración, y `reiniciar` que restaura configuración inicial posicionando pelota en centro del campo con velocidad aleatoria.

La clase `Paleta` representa componente hoja con atributos `ancho` y `alto` que definen dimensiones del rectángulo dibujado, `rapidez` que determina velocidad de desplazamiento vertical, `color` que especifica apariencia, `tieneEspinass` que indica si ítem temporal de daño está activo, `estrategiaMovimiento` que encapsula algoritmo de control intercambiable, y `estadoOriginal` que almacena configuración previa a aplicación de ítems permitiendo restauración. El método `actualizar` delega cálculo de dirección de movimiento a estrategia registrada que retorna enumeración `ARRIBA`, `ABAJO` o `NINGUNA` según análisis de estado del juego, ajustando posteriormente posición vertical respetando límites de campo para prevenir desplazamiento fuera de área válida. El método `mover` encapsula modificación de posición verificando restricciones de frontera. La clase proporciona operaciones para gestión de ítems como `agregarEspinass` y `eliminarEspinass` que modifican atributo booleano, `redimensionar` que ajusta dimensiones aplicando factor multiplicativo, `clonar` que retorna copia profunda preservando configuración completa, y `guardarEstado` con `restaurarEstado` que implementan memento para reversión de transformaciones temporales.

La clase `Bloque` representa componente hoja con atributos `ancho`, `alto`, `destruible` que indica si puede ser eliminado mediante impactos, `salud` que cuantifica impactos requeridos para destrucción, `color` y `puntos` otorgados al jugador al destruir. El método `golpear` decrementa salud retornando booleano indicando si bloque fue destruido facilitando remoción de colección de elementos activos. El método `obtenerPuntos` retorna valor asociado utilizado por modelo para incrementar puntaje del jugador.

La clase `ObjetoJuegoCompuesto` implementa el nodo compuesto del patrón mediante gestión de colección interna hijos que almacena referencias a `ComponenteJuego` subordinados, proporcionando métodos agregar y eliminar para manipulación dinámica de composición. El método `obtenerHijos` retorna copia defensiva de lista previniendo modificaciones externas no controladas. Los métodos `actualizar` y `dibujar` implementan procesamiento recursivo invocando operación correspondiente en cada hijo mediante iteración sobre colección, delegando comportamiento específico a componentes subordinados sin conocimiento de tipos concretos. El método `obtenerLimites` calcula caja delimitadora que engloba todos los hijos mediante cálculo de mínimos y máximos de coordenadas, retornando `Rectangle2D` que representa área ocupada por composición completa.

La integración del patrón en el sistema permite que `ModeloJuego` gestione colección heterogénea de objetos mediante referencias polimórficas a `ComponenteJuego`, simplificando bucle de actualización que itera elementos invocando `actualizar` sin condicionales basados en tipo. Similarmente, el renderizado procesa todos los objetos uniformemente mediante invocación a `dibujar` delegando detalles de representación a implementaciones específicas. Esta abstracción facilita extensión con nuevos tipos de objetos mediante implementación de `ComponenteJuego` sin modificar lógica central de procesamiento, adhiriendo al principio abierto-cerrado que maximiza estabilidad de código existente mientras permite crecimiento funcional.

4.3. Patrón Strategy - Sistema de Colisiones

El patrón Strategy fue implementado en el sistema de colisiones para encapsular algoritmos de detección y resolución en clases intercambiables independientes, permitiendo que gestor seleccione dinámicamente estrategia apropiada según tipos de objetos involucrados sin lógica condicional compleja en código cliente. Esta elección responde a la necesidad de procesar colisiones mediante algoritmos distintos dependiendo de si pelota impacta paleta requiriendo cálculo de ángulo de rebote, pelota impacta bloque necesitando destrucción y generación probabilística de item, o pelota impacta pared provocando rebote simple con inversión de componente de velocidad perpendicular.

La interfaz `EstrategiaColision` define contrato común mediante métodos `manejarColision` que ejecuta lógica de resolución modificando estado de objetos involucrados, y `verificarColision` que retorna booleano indicando si geometrías de objetos proporcionados intersectan detectando potencial colisión. Esta abstracción permite que nuevas estrategias sean agregadas sin modificar código del gestor que las utiliza, facilitando extensión futura con comportamientos adicionales como colisiones entre múltiples pelotas que fusionan o items que interaccionan con bloques aplicando efectos especiales.

La clase `GestorColisiones` actúa como contexto que mantiene registro de estrategias disponibles almacenadas en Map que asocia clave identificadora con instancia de `EstrategiaColision` correspondiente. El método `registrarEstrategia` permite agregar nuevas estrategias dinámicamente durante inicialización del sistema, configurando catálogo completo de manejadores especializados. El método `verificarTodasColisiones` implementa bucle que itera pares de objetos del juego invocando `verificarColision` en estrategia apropiada seleccionada mediante análisis de tipos de objetos, ejecutando `manejarColision` cuando detección retorna verdadero indicando intersección. El método `manejarColision` proporciona interfaz simplificada que encapsula selección de estrategia y delegación de procesamiento, ocultando complejidad de registro interno al código cliente que simplemente proporciona objetos involucrados.

La clase `EstrategiaColisionPelotaPaleta` implementa manejo especializado de impactos entre pelota y paleta calculando ángulo de rebote basado en punto de contacto relativo a centro de paleta, permitiendo que jugador controle dirección de pelota mediante posicionamiento preciso creando jugabilidad táctica donde intercepciones en extremos de paleta producen ángulos pronunciados que dificultan defensa del oponente. El método `verificarColision` invoca `intersects` entre cajas delimitadoras de pelota y paleta retornando resultado de comparación geométrica. El método `manejarColision` calcula punto de impacto restando coordenada Y de pelota de coordenada Y de paleta, normalizando resultado mediante división por alto de paleta para obtener valor en rango menos uno a uno donde cero representa impacto en centro y extremos representan impactos en bordes superior e inferior. El método privado `calcularAnguloRebote` traduce punto de impacto normalizado a ángulo en radianes mediante función lineal que mapea rango de entrada a rango de ángulos válidos, aplicando posteriormente componentes trigonométricos para actualizar velocidades de pelota preservando magnitud mientras modifica dirección. Adicionalmente, la

estrategia verifica si paleta tiene espinas activas, aplicando penalización al puntaje del jugador controlador de paleta impactada cuando condición se cumple, implementando mecánica de riesgo-recompensa donde items ofensivos generan ventaja pero penalizan errores defensivos.

La clase EstrategiaColisionPelotaBloque implementa destrucción de bloques mediante invocación a método golpear que decrementa salud retornando booleano indicando si bloque fue completamente destruido. Cuando destrucción ocurre, estrategia invoca método privado generarItem que aplica probabilidad configurable determinando si item temporal debe ser creado en posición del bloque destruido, agregando instancia de Item apropiada a colección gestionada por GestorItems cuando generación es exitosa. La estrategia incrementa puntaje del jugador que provocó destrucción mediante adición de puntos asociados al bloque, recompensando eliminación de formaciones complejas. Adicionalmente, estrategia invierte componente apropiado de velocidad de pelota simulando rebote físico que preserva energía cinética mientras modifica trayectoria.

La clase EstrategiaColisionPelotaPared implementa rebote simple contra fronteras del campo de juego mediante inversión de componente de velocidad perpendicular a pared impactada, preservando componente paralela que mantiene dirección general de movimiento. La estrategia verifica coordenadas de pelota comparando con límites superior, inferior, izquierdo y derecho del campo, invirtiendo velocidadY cuando pelota alcanza fronteras verticales y velocidadX cuando atraviesa fronteras horizontales simulando victoria de jugador opuesto. Esta lógica implementa condición de derrota donde pelota que atraviesa línea defensiva propia incrementa puntaje enemigo y reposiciona pelota en centro con velocidad aleatoria reiniciando jugada.

La integración del patrón en el sistema permite que lógica de colisiones evolucione independientemente del resto del modelo mediante adición o modificación de estrategias sin impactar bucle principal de actualización que delega procesamiento a gestor. Esta separación de algoritmos en clases especializadas facilita pruebas unitarias que verifican comportamiento de cada estrategia aisladamente proporcionando casos de entrada controlados y validando modificaciones de estado resultantes. La arquitectura permite configuración dinámica de estrategias durante ejecución habilitando modos de juego alternativos con mecánicas de colisión modificadas sin requerir compilación de versiones distintas del ejecutable.

4.4. Patrón Strategy - Sistema de Movimiento e Inteligencia Artificial

El patrón Strategy fue implementado adicionalmente en el sistema de movimiento de paletas para encapsular algoritmos de control en clases intercambiables que calculan dirección de desplazamiento apropiada según análisis del estado del juego. Esta elección se fundamenta en la necesidad de soportar paletas controladas por jugador humano mediante entrada de teclado y paletas controladas por inteligencia artificial con múltiples niveles de dificultad que requieren algoritmos de predicción con complejidades variables.

La interfaz EstrategiaMovimiento define contrato común mediante método calcularMovimiento que recibe referencia a paleta que requiere actualización, pelota cuya trayectoria debe ser considerada, y delta temporal para compensar variaciones en velocidad de procesamiento, retornando enumeración Direccion con valores ARRIBA, ABAJO o NINGUNA que indican desplazamiento deseado. Esta abstracción permite que Paleta delegue cálculo de movimiento a estrategia registrada sin conocimiento del algoritmo específico implementado, facilitando intercambio dinámico de comportamiento sin modificar clase host.

La clase EstrategiaMovimientoJugador implementa control mediante entrada de usuario manteniendo referencia a AdaptadorEntrada que encapsula estado actual del teclado con teclas presionadas registradas en mapa consultable. El método calcularMovimiento verifica si teclas de dirección asignadas están presionadas retornando ARRIBA cuando tecla superior está activa, ABAJO cuando tecla inferior está activa, y NINGUNA cuando ninguna o ambas están presionadas simulando cancelación de fuerzas opuestas. Esta lógica proporciona control directo y responsivo permitiendo que jugador posicione paleta precisamente mediante mantenimiento de teclas apropiadas.

La clase EstrategiaMovimientoIAMedio implementa inteligencia artificial con complejidad intermedia mediante algoritmo que predice posición futura de pelota considerando velocidad actual y tiempo

estimado hasta intersección con línea defensiva. El atributo `factorPrediccion` configura agresividad de predicción donde valores altos anticipan trayectoria futura compensando latencia de respuesta y valores bajos priorizan seguimiento conservador de posición actual minimizando errores de predicción cuando pelota cambia dirección inesperadamente. El método `calcularMovimiento` calcula tiempo de intersección dividiendo distancia horizontal entre pelota y paleta por magnitud de velocidad horizontal de pelota, multiplicando resultado por velocidad vertical para obtener desplazamiento vertical esperado durante trayecto. La estrategia suma desplazamiento predicho a posición vertical actual de pelota obteniendo coordenada objetivo, comparando con posición de paleta para determinar si debe desplazarse hacia arriba o abajo alcanzando posición de intercepción. Esta lógica proporciona oponente desafiante que anticipa movimientos requiriendo que jugador humano utilice tácticas de engaño como rebotes en ángulos extremos que invalidan predicciones lineales.

La enumeración `Direccion` proporciona abstracción type-safe para representar dirección de movimiento evitando uso de constantes mágicas enteras o booleanos que dificultan comprensión de semántica. Los valores `ARRIBA`, `ABAJO` y `NINGUNA` expresan explícitamente intención de movimiento facilitando lectura de código que manipula resultado de estrategias.

La clase `FabricaIA` encapsula lógica de instanciación de estrategias de inteligencia artificial proporcionando método público `crearIA` que recibe enumeración `DificultadIA` retornando implementación apropiada de `EstrategiaMovimiento`. Los métodos privados `crearIAFacil`, `crearIAMedio` y `crearIADifícil` instancian estrategias concretas con configuraciones optimizadas incluyendo factores de predicción ajustados según nivel deseado. Esta separación de responsabilidades permite que código cliente solicite inteligencia artificial especificando únicamente dificultad sin conocimiento de clases concretas involucradas o parámetros de configuración requeridos, simplificando uso y centralizando decisiones de diseño en factoría que puede evolucionar independientemente.

La clase `ServicioIA` actúa como fachada que simplifica interacción con sistema de inteligencia artificial manteniendo referencia a `FabricaIA` y dificultad actual configurada. El método `establecerDificultad` permite modificación dinámica de nivel creando nueva estrategia mediante invocación a factoría. El método `obtenerEstrategiaMovimiento` retorna estrategia actual permitiendo asignación a paleta controlada por computadora. El método `calcularSiguienteMovimiento` encapsula delegación a estrategia proporcionando interfaz conveniente que acepta parámetros necesarios y retorna dirección calculada.

La enumeración `DificultadIA` define niveles `FACIL`, `MEDIO` y `DIFÍCIL` que representan complejidad de algoritmos disponibles, proporcionando abstracción semántica que mejora expresividad de código cliente que configura oponente artificial.

La integración del patrón permite que sistema soporte múltiples implementaciones de inteligencia artificial sin proliferación de condicionales en `Paleta` que de otro modo requeriría distinciones explícitas basadas en tipo de control. La arquitectura facilita extensión con nuevas estrategias como redes neuronales entrenadas mediante aprendizaje por refuerzo o algoritmos genéticos que evolucionan comportamiento, implementables como `EstrategiaMovimiento` adicionales registrables en factoría sin modificar clases existentes. La separación de algoritmo de contexto permite pruebas unitarias que verifican lógica de predicción proporcionando estados de juego sintéticos validando direcciones calculadas sin requerir simulación completa de partida.

4.5. Patrón Factory - Sistema de Inteligencia Artificial

El patrón Factory fue implementado en la creación de estrategias de inteligencia artificial para encapsular lógica de instanciación en clase especializada que oculta detalles de construcción y dependencias de implementaciones concretas al código cliente. Esta elección se fundamenta en la necesidad de centralizar decisiones de configuración como factores de predicción, umbrales de reacción y agresividad de comportamiento que varían según nivel de dificultad deseado, evitando que código cliente deba conocer parámetros de inicialización específicos de cada estrategia concreta.

La clase `FabricaIA` proporciona método público estático `crearIA` que recibe enumeración `DificultadIA` especificando nivel deseado, delegando instanciación a método privado apropiado mediante evaluación

de switch que selecciona comportamiento según valor de entrada. El método `crearIAFacil` instancia `EstrategiaMovimientoIAFacil` configurando factor de predicción bajo que prioriza seguimiento conservador de posición actual de pelota, minimizando anticipación para simular reacción lenta apropiada para jugadores principiantes que requieren oponente predecible permitiendo aprendizaje de mecánicas básicas. El método `crearIAMedio` instancia `EstrategiaMovimientoIAMedio` con factor de predicción intermedio que balancea anticipación de trayectoria futura con estabilidad ante cambios de dirección, proporcionando desafío moderado para jugadores que dominan controles pero aún desarrollan tácticas avanzadas. El método `crearIADifícil` instancia `EstrategiaMovimientoIADifícil` con factor de predicción alto que maximiza anticipación calculando trayectorias complejas considerando múltiples rebotes futuros, simulando oponente prácticamente infalible que requiere tácticas sofisticadas de engaño para superar.

La encapsulación de lógica de creación permite que parámetros de configuración evolucionen sin impactar código cliente que simplemente especifica dificultad deseada recibiendo objeto funcionalmente completo. Si ajustes de balance durante fase de pruebas determinan que factor de predicción medio requiere reducción para mejorar jugabilidad, modificación se realiza únicamente en método `crearIAMedio` de la factoría sin afectar clases que utilizan estrategias creadas. Esta centralización de responsabilidad de inicialización facilita mantenimiento y ajuste de parámetros que determinan características comportamentales de inteligencia artificial.

La integración del patrón con `ServicioIA` que actúa como fachada proporciona interfaz conveniente donde código cliente invoca `establecerDificultad` especificando nivel, desencadenando creación automática de estrategia apropiada mediante delegación a factoría. El servicio almacena estrategia creada permitiendo consultas posteriores mediante `obtenerEstrategiaMovimiento` que retorna referencia asignable a paleta controlada por computadora. Esta arquitectura bicapa donde servicio coordina interacciones con factoría simplifica uso del sistema de inteligencia artificial reduciendo complejidad visible al código cliente que configura oponentes artificiales.

4.6. Patrón Factory - Sistema de Generación de Niveles

El patrón Factory fue implementado adicionalmente en la generación de niveles predefinidos para encapsular lógica de creación de formaciones de bloques optimizadas que proporcionan desafío apropiado según experiencia del jugador. Esta elección responde a la necesidad de proporcionar catálogo de mapas probados que jugadores pueden seleccionar sin requerir diseño manual mediante editor, garantizando calidad de experiencia mediante configuraciones balanceadas que evitan frustración por dificultad excesiva o aburrimiento por simplicidad inadecuada.

La clase `FabricaMapasPredefinidos` proporciona método público `crearNivel` que recibe identificador de nivel deseado delegando instanciación a método privado especializado según valor proporcionado. El método `crearNivelClasico` instancia `Nivel` con nombre descriptivo, dificultad baja y formación simple de bloques destructibles organizados en filas horizontales con espaciado regular, proporcionando introducción accesible a modo Breakout donde jugador desarrolla habilidades de control de pelota sin presión de patrones complejos. El método `crearNivelIntermedio` configura nivel con dificultad media incorporando mezcla de bloques destructibles e indestructibles organizados en patrón que requiere planeación táctica para alcanzar bloques protegidos, incentivando dominio de control de ángulos de rebote. El método `crearNivelDifícil` genera nivel avanzado con dificultad alta incluyendo formaciones densas con bloques de múltiples golpes que requieren impactos repetidos para destrucción, bloques indestructibles que obstruyen trayectorias directas forzando rebotes complejos, y distribución que maximiza tiempo requerido para completar nivel desafiando persistencia del jugador.

La clase `Nivel` encapsula configuración completa de mapa con atributos `id` que identifica nivel únicamente, nombre descriptivo visualizado en interfaz de selección, dificultad que categoriza desafío esperado, colección bloques que almacena instancias de `Bloque` con posiciones y propiedades especificadas, `tiempoLimite` opcional que establece duración máxima para completar nivel agregando presión temporal, y `bandera bloqueados` que indica si nivel requiere desbloquearse mediante completar niveles previos. La clase proporciona métodos `agregarBloque` y `eliminarBloque` para manipulación dinámica de formación, `estaCompletado` que verifica si todos los bloques destructibles han sido eliminados determinando victoria, y `cargar` con `guardar` que serializan configuración en formato persistente permitiendo distribución de niveles personalizados creados por comunidad.

La clase `ServicioNiveles` coordina acceso a niveles predefinidos y personalizados manteniendo referencia a `FabricaMapasPredefinidos` y colección `nivelesPersonalizados` que almacena niveles creados por usuario mediante editor. El método `cargarNivel` acepta identificador verificando primero si corresponde a nivel predefinido delegando creación a factoría, o consultando colección de personalizados cuando identificador no coincide con predefinidos, retornando instancia de `Nivel` lista para uso. El método `guardarNivelPersonalizado` serializa configuración proporcionada por usuario persistiendo en sistema de archivos, agregando posteriormente a colección de personalizados para disponibilidad inmediata sin requerir reinicio de aplicación.

La integración del patrón permite expandir catálogo de niveles predefinidos mediante adición de métodos de creación en factoría sin impactar código cliente que consulta niveles mediante servicio. La separación entre factoría que proporciona predefinidos y servicio que coordina acceso unificado a predefinidos y personalizados simplifica arquitectura manteniendo responsabilidades especializadas en clases dedicadas que evolucionan independientemente.

4.7. Patrón Template Method

El patrón `Template Method` fue implementado en el sistema de modos de juego para definir esqueleto de algoritmo de ejecución en clase abstracta con pasos especializables que subclases concretas implementan según semántica específica de cada modalidad. Esta elección se fundamenta en la necesidad de reutilizar lógica común como inicialización de modelo con posicionamiento de elementos, bucle principal de actualización que procesa física y colisiones, y finalización con persistencia de puntajes, mientras permite personalización de verificaciones de victoria y derrota que varían según reglas de cada modo.

La clase abstracta `ModoJuegoAbstracto` define método `template jugar` que implementa secuencia completa de ejecución invocando métodos gancho en orden apropiado. La secuencia inicia con invocación a `inicializarJuego` que subclases especializan para configurar estado inicial específico del modo incluyendo número de pelotas activas, formaciones de bloques, y condiciones de victoria aplicables. Posteriormente, el `template` entra en bucle principal que itera mientras `verificarCondicionVictoria` y `verificarCondicionDerrota` retornan falso, invocando en cada iteración `actualizarJuego` que delega a subclase procesamiento especializado como actualización de temporizadores, verificación de items activos, o gestión de múltiples pelotas. Al detectar cumplimiento de condición de terminación, el `template` invoca `finalizarJuego` permitiendo que subclase ejecute lógica de limpieza como persistencia de puntaje final, desbloqueo de logros, o generación de replay.

Los métodos gancho proporcionan puntos de extensión donde subclases concretas inyectan comportamiento especializado sin duplicar lógica del bucle principal. El método `inicializarJuego` configura estado inicial del modelo posicionando elementos según requisitos del modo, por ejemplo en modo `Breakout` creando formación de bloques mediante invocación a nivel cargado mientras modo `Clásico` omite bloques focalizando exclusivamente en paletas y pelota. El método `actualizarJuego` proporciona punto de inserción para procesamiento adicional ejecutado en cada frame, permitiendo por ejemplo actualización de temporizador en modo con tiempo límite o gestión de múltiples pelotas en modo caótico. El método `verificarCondicionVictoria` implementa lógica específica que determina si jugador alcanzó objetivo del modo, retornando verdadero cuando puntaje alcanza umbral en modo por puntos o todos los bloques destructibles han sido eliminados en modo `Breakout`. El método `verificarCondicionDerrota` implementa verificación complementaria que retorna verdadero cuando jugador pierde, por ejemplo agotando vidas disponibles en modo `Breakout` o permitiendo que oponente alcance umbral de puntaje en modo `Clásico`. El método `finalizarJuego` ejecuta limpieza persistiendo resultado en sistema de puntajes altos, notificando observadores sobre terminación para actualización de interfaz, y liberando recursos temporales.

La clase `ModoClasico` extiende plantilla abstracta implementando semántica tradicional de `Pong` uno contra uno donde victoria se alcanza cuando jugador acumula puntaje máximo configurado mientras derrota ocurre cuando oponente alcanza mismo umbral. El atributo `puntajeMaximo` establece objetivo personalizable durante inicialización del modo. La implementación de `inicializarJuego` posiciona paletas en extremos izquierdo y derecho del campo, crea pelota única en centro con velocidad aleatoria, y configura paleta enemiga con estrategia de inteligencia artificial según dificultad seleccionada. La implementación de `verificarCondicionVictoria` consulta puntaje actual del jugador comparando con

umbral configurado. La implementación de `verificarCondicionDerrota` consulta puntaje del oponente aplicando misma comparación. La implementación de `finalizarJuego` persiste resultado en historial de puntajes asociado a cuenta de usuario autenticado.

La clase `ModoBreakout` extiende plantilla implementando semántica de destrucción de bloques donde jugador controla paleta inferior reflejando pelota hacia formación superior, acumulando puntaje mediante eliminación de bloques mientras evita que pelota caiga por borde inferior provocando pérdida de vida. El atributo `vidasRestantes` cuantifica número de errores permitidos antes de derrota definitiva. La implementación de `inicializarJuego` posiciona paleta única en parte inferior central, carga nivel seleccionado creando formación de bloques mediante invocación a `ServicioNiveles`, y crea pelota adherida a paleta esperando liberación mediante input del jugador. La implementación de `actualizarJuego` detecta cuando pelota atraviesa borde inferior decrementando `vidasRestantes` y reposicionando pelota adherida a paleta para continuar juego, o invocando `finalizarJuego` cuando vidas se agotan. La implementación de `verificarCondicionVictoria` consulta nivel verificando mediante `estaCompletado` si todos los bloques destructibles han sido eliminados. La implementación de `verificarCondicionDerrota` retorna verdadero cuando `vidasRestantes` alcanza cero. La implementación de `finalizarJuego` persiste puntaje final y actualiza progreso de nivel desbloqueando siguiente mapa si completar exitosamente.

La integración del patrón permite incorporar nuevos modos de juego mediante extensión de plantilla abstracta sin duplicación de lógica compartida, facilitando crecimiento del sistema con modalidades adicionales como modo multijugador cooperativo, modo contrarreloj o modo supervivencia con oleadas progresivas. La centralización del algoritmo principal en `template` permite refinamiento de bucle de juego como optimización de rendimiento o integración de sistema de pausa afectando automáticamente todos los modos sin requerir modificaciones individuales en cada subclase.

4.8. Patrón Builder

El patrón Builder fue implementado en el sistema de construcción de niveles personalizados para separar algoritmo complejo de especificación incremental de propiedades de la representación final del producto, permitiendo que usuarios diseñen mapas mediante interfaz fluida que configura paso a paso nombre, dificultad y formación de bloques. Esta elección responde a la necesidad de simplificar proceso de creación de niveles con múltiples atributos opcionales que pueden especificarse en orden arbitrario, evitando constructores con listas extensas de parámetros que dificultan comprensión y facilitan errores de invocación.

La clase `ConstructorMapa` implementa interfaz fluida mediante métodos que retornan referencia a la instancia permitiendo encadenamiento de invocaciones, manteniendo internamente instancia parcialmente construida de `Nivel` y colección de bloques acumulados durante especificación. El método `reiniciar` crea nueva instancia de `Nivel` descartando progreso previo, permitiendo reutilización del constructor para múltiples creaciones. El método `establecerNombre` asigna identificador textual descriptivo al nivel, retornando `this` para continuar encadenamiento. El método `establecerDificultad` configura categoría de desafío mediante asignación de enumeración que posteriormente influye en selección de niveles por jugadores según experiencia.

El método `agregarBloque` proporciona interfaz genérica que acepta instancia de `Bloque` completamente configurada, agregando elemento a colección interna que posteriormente se asignará a nivel. El método `agregarBloqueDestructible` simplifica creación de bloque común instanciando `Bloque` con parámetros de posición, dimensiones, atributo destructible en verdadero y salud predeterminada, agregando resultado a colección mediante invocación a `agregarBloque`. El método `agregarBloqueIndestructible` proporciona conveniencia similar creando bloque con atributo destructible en falso y salud configurada a valor alto simulando invulnerabilidad. El método `agregarPatronBloques` acepta especificación de formación como matriz bidimensional de enumeraciones `TipoBloque`, iterando elementos creando bloques apropiados en posiciones calculadas según índices de iteración, facilitando diseño de patrones complejos mediante especificación declarativa compacta.

La enumeración `TipoBloque` proporciona abstracción que categoriza bloques según propiedades semánticas incluyendo `DESTRUCTIBLE` para bloques eliminables con salud estándar, `INDESTRUCTIBLE` para bloques permanentes que obstruyen trayectorias, `BONUS` para bloques que incrementan probabilidad de

generación de items, y MULTI_GOLPE para bloques con salud elevada requiriendo impactos múltiples. Esta abstracción permite que método `agregarPatronBloques` interprete matriz de tipos instanciando bloques con configuraciones apropiadas según categoría especificada.

El método `construir` finaliza proceso de especificación ejecutando validaciones de consistencia que verifican que nombre ha sido establecido y al menos un bloque ha sido agregado, lanzando excepción cuando precondiciones no se satisfacen previniendo creación de niveles parcialmente configurados. Superadas validaciones, método asigna colección acumulada de bloques al nivel, retorna instancia completamente inicializada, e invoca `reiniciar` preparando constructor para creación subsecuente. Este diseño garantiza que producto retornado se encuentra en estado válido apto para uso inmediato.

La integración del patrón en el editor de mapas proporciona experiencia de usuario fluida donde especificación de nivel se expresa mediante secuencia legible de invocaciones como `constructor.establecerNombre("Mi Nivel").establecerDificultad(DificultadIA.MEDIO).agregarPatronBloques(matriz).construir()`, mejorando expresividad comparado con constructor tradicional que requeriría especificar parámetros posicionales incluyendo valores nulos para opcionales no deseados. La arquitectura permite extensión del builder con métodos adicionales como `establecerTiempoLimite` o `configurarGravedad` sin impactar código existente que no utiliza nuevas capacidades, facilitando evolución del sistema de niveles incorporando propiedades adicionales conforme requisitos evolucionen.

4.9. Patrón Prototype

El patrón Prototype fue implementado en el sistema de personalización de paletas para permitir creación rápida de configuraciones basadas en prototipos predefinidos que sirven como plantillas clonables, evitando especificación repetitiva de propiedades comunes cuando variaciones entre configuraciones son mínimas. Esta elección responde a la necesidad de proporcionar catálogo de paletas preconfiguradas con combinaciones probadas de dimensiones, velocidades y colores que jugadores pueden adoptar directamente o modificar incrementalmente según preferencias individuales.

La clase `PrototipoPaleta` actúa como registro que gestiona colección de configuraciones predefinidas almacenadas en Map que asocia identificador textual con instancia de `Paleta` que sirve como prototipo. El método `registrarPrototipo` permite agregar nuevas configuraciones al catálogo especificando nombre y paleta prototipo completamente inicializada, almacenando referencia en mapa interno. El método `clonarPaleta` recupera prototipo identificado por nombre consultando mapa, invocando posteriormente método `clonar` de `Paleta` que retorna copia profunda preservando configuración completa sin compartir referencias mutables con original. El método `crearPaletaPersonalizada` acepta configuración de personalización especificada mediante objeto `ConfigPaleta`, clonando prototipo base y aplicando modificaciones especificadas mediante invocación a método `aplicarA` que ajusta propiedades según preferencias, retornando paleta resultante lista para uso.

La clase `ConfigPaleta` encapsula conjunto de propiedades personalizables incluyendo ancho y alto que definen dimensiones geométricas, rapidez que determina velocidad de desplazamiento vertical, y colores primario y secundario que especifican apariencia visual mediante tonos alternantes que crean efecto visual distintivo. El método `aplicarA` recibe referencia a `Paleta` modificando atributos mediante invocación a setters apropiados, encapsulando lógica de aplicación de configuración en método centralizado reusable.

La clase `Paleta` implementa método `clonar` que proporciona capacidad de replicación profunda instanciando nueva `Paleta` mediante invocación a constructor que acepta parámetros de configuración extraídos de instancia actual, retornando copia independiente que preserva valores de atributos sin compartir referencias a objetos mutables. El método debe garantizar independencia completa entre original y clon para prevenir efectos laterales donde modificaciones a clon afecten inadvertidamente prototipo registrado corrompiendo catálogo de configuraciones predefinidas.

El singleton `GestorPrototiposPaleta` coordina acceso global a sistema de prototipos manteniendo instancia única de `PrototipoPaleta` junto con referencia a configuraciones predefinidas. El método `obtenerInstancia` retorna singleton aplicando inicialización perezosa. El método `registrarPrototiposDefecto` se invoca durante inicialización del sistema creando paletas predefinidas con nombres como `Clasica`, `Rapidaz`

"Grande", registrando cada una mediante invocación a `PrototipoPaleta.registrarPrototipo`. El método `obtenerPrototipo` delega consulta a `PrototipoPaleta` retornando clon de configuración solicitada. El método `guardarPrototipoUsuario` permite que jugador registre configuración personalizada como prototipo reutilizable, persistiendo adicionalmente en sistema de archivos para disponibilidad entre sesiones. El método `listarPrototiposDisponibles` retorna conjunto de nombres de prototipos registrados facilitando presentación de opciones en interfaz de selección.

La integración del patrón en el sistema de personalización proporciona experiencia donde jugador selecciona prototipo base de lista de opciones predefinidas, visualiza paleta resultante en preview, y opcionalmente ajusta propiedades individuales mediante controles especializados como sliders para dimensiones y selectores de color para apariencia. El sistema clona prototipo seleccionado aplicando modificaciones sobre copia independiente que puede guardarse como configuración permanente asociada a cuenta de usuario o registrarse como prototipo nuevo compartible con comunidad. Esta aproximación optimiza experiencia evitando especificación completa desde cero cuando configuración deseada difiere marginalmente de prototipo existente.

4.10. Patrón Singleton

El patrón Singleton fue implementado en tres clases fundamentales del sistema que requieren instancia única compartida globalmente para coordinar estado común sin proliferación de objetos redundantes que complicarían sincronización. Esta elección se fundamenta en la naturaleza centralizada de gestión de puntajes altos persistidos entre sesiones, configuración global de aplicación compartida por todos los subsistemas, y registro de prototipos de paletas accesible universalmente.

La clase `GestorPuntajes` implementa Singleton mediante constructor privado que previene instanciación externa, atributo estático privado `instancia` que almacena referencia única, y método estático público `obtenerInstancia` que controla acceso aplicando inicialización perezosa que crea instancia únicamente durante primera invocación. La clase gestiona colección `puntajesAltos` que almacena registros históricos ordenados por puntuación descendente, y `sesionActual` que mantiene información de partida en progreso. El método `agregarPuntaje` inserta nuevo registro en colección verificando si puntuación califica para tabla de posiciones, invocando posteriormente `servicioPersistencia` para almacenar cambios en sistema de archivos garantizando durabilidad. El método `obtenerPuntajesAltos` retorna copia defensiva de tabla previniendo modificaciones externas no controladas. El método `guardarPuntajes` delega persistencia a servicio especializado serializando colección en formato JSON. El método `cargarPuntajes` restaura estado desde almacenamiento durante inicialización de aplicación. El método `actualizarSesionActual` modifica información de partida en progreso reflejando incrementos de puntaje o eventos relevantes.

La clase `ConfiguracionJuego` implementa Singleton gestionando parámetros globales incluyendo `anchoPantalla` y `altoPantalla` que determinan resolución de ventana de juego, `rapidezPelota` y `rapidezPaleta` que establecen velocidades base antes de modificaciones por items, y volumen que controla nivel de audio de efectos sonoros y música. El constructor privado inicializa atributos con valores predeterminados apropiados. El método `obtenerInstancia` proporciona acceso controlado aplicando inicialización perezosa. El método `cargarConfiguracion` lee preferencias almacenadas desde sistema de archivos restaurando configuración establecida por usuario en sesión previa. El método `guardarConfiguracion` persiste estado actual permitiendo durabilidad de preferencias entre ejecuciones. El método `establecerVolumen` modifica nivel de audio invocando notificación a subsistema de reproducción sonora que ajusta ganancia apropiadamente. Los métodos getters proporcionan acceso de lectura a parámetros permitiendo que subsistemas consulten configuración sin capacidad de modificación no autorizada.

La clase `GestorPrototiposPaleta` implementa Singleton coordinando acceso a sistema de prototipos mediante constructor privado, atributo estático `instancia` y método `obtenerInstancia` con inicialización perezosa. La clase mantiene referencia a objeto prototipos de tipo `PrototipoPaleta` que gestiona registro efectivo. El método `registrarPrototiposDefecto` invoca durante inicialización creando paletas predefinidas que pueblan catálogo inicial. El método `obtenerPrototipo` delega consulta a objeto prototipos retornando clon de configuración solicitada. El método `guardarPrototipoUsuario` permite registro de configuraciones personalizadas delegando almacenamiento a prototipos con persistencia adicional en sistema de archivos. El método `listarPrototiposDisponibles` delega enumeración de nombres registrados facilitando presentación en interfaz.

La integración del patrón garantiza que múltiples subsistemas acceden a instancias consistentes de gestores sin posibilidad de desincronización que ocurriría con múltiples instancias independientes gestionando datos supuestamente globales. El Singleton facilita acceso conveniente desde cualquier punto del sistema mediante invocación estática a `obtenerInstancia` sin requerir pasaje explícito de referencias a través de cadenas de invocación profundas. La inicialización perezosa optimiza uso de recursos posponiendo creación hasta que funcionalidad es efectivamente requerida, evitando overhead de inicialización de componentes no utilizados durante sesión particular.

4.11. Patrón Decorator

El patrón Decorator fue implementado en el sistema de usuarios para agregar dinámicamente capacidad de gestión de logros sin modificar clase base, permitiendo composición flexible donde funcionalidad de logros puede aplicarse selectivamente a usuarios que participan en modos de juego que soportan desbloqueo de reconocimientos. Esta elección se fundamenta en la necesidad de extender comportamiento de objetos `Usuario` sin proliferación de subclases que combinen ortogonalmente características opcionales como logros, estadísticas detalladas e integración con redes sociales.

La interfaz `ComponenteUsuario` define contrato abstracto estableciendo operaciones fundamentales que todos los componentes de usuario deben implementar, incluyendo `obtenerDescripcion` que retorna representación textual completa, `obtenerNivel` que cuantifica progresión acumulada, y `obtenerExperiencia` que retorna puntos de experiencia ganados mediante juego. Esta abstracción permite que decoradores y componentes concretos sean tratados uniformemente mediante referencias polimórficas, facilitando encadenamiento transparente de decoradores sin que código cliente distinga entre usuario decorado y no decorado.

La clase `Usuario` implementa `ComponenteUsuario` proporcionando implementación concreta con atributos `id` que identifica usuario únicamente, `nombreUsuario` y `clave` para autenticación, `correo` para comunicaciones, `nivel` y `experiencia` que cuantifican progresión, `fechaCreacion` que registra antigüedad de cuenta, y `configPaletaPreferida` que almacena personalización visual predeterminada. El método `obtenerDescripcion` retorna cadena formateada incluyendo nombre, nivel y experiencia. El método `agregarExperiencia` incrementa contador aplicando posteriormente verificación de umbral para promoción de nivel cuando acumulación supera valor configurado. El método `guardarConfiguracionPaleta` persiste preferencia de personalización asociando configuración con cuenta. El método `obtenerConfiguracionPaleta` retorna configuración almacenada permitiendo aplicación automática durante inicialización de partida.

La clase abstracta `DecoradorUsuario` implementa `ComponenteUsuario` manteniendo referencia a `ComponenteUsuario` decorado denominada `usuarioDecorado`, estableciendo relación de composición que permite encadenamiento recursivo. Los métodos de interfaz proporcionan implementaciones por defecto que delegan invocación al componente decorado mediante forwarding simple, preservando comportamiento original mientras permiten que subclases concretas sobrescriban métodos específicos para inyectar funcionalidad adicional. Esta estructura facilita reutilización de lógica de delegación evitando duplicación en decoradores concretos que solamente necesitan especializar métodos específicos.

La clase `DecoradorLogros` extiende decorador abstracto agregando gestión de reconocimientos mediante atributo `logros` que almacena colección de instancias de `Logro` asociadas. El método `agregarLogro` inserta nuevo logro en colección verificando que no haya sido previamente agregado previniendo duplicados. El método `obtenerLogros` retorna copia defensiva de colección permitiendo consulta sin riesgo de modificaciones externas. El método `obtenerDescripcion` sobrescribe implementación heredada invocando `obtenerDescripcion` en usuario decorado para obtener descripción base, concatenando posteriormente información de logros desbloqueados mediante iteración sobre colección formateando cada elemento, retornando cadena combinada que incluye información original extendida con reconocimientos. Los métodos `obtenerNivel` y `obtenerExperiencia` delegan sin modificación preservando comportamiento original.

La clase `Logro` encapsula información de reconocimiento individual con atributos `id`, `nombre` descriptivo, descripción detallada explicando condición de desbloqueo, `icono` que referencia recurso gráfico representativo, y `desbloqueadoEn` que registra timestamp de obtención. El método `desbloquear` marca logro como obtenido asignando timestamp actual. El método `estaDesbloqueado` retorna booleano verificando

si timestamp ha sido establecido.

La integración del patrón en el sistema de autenticación permite que `ServicioAutenticacion` decore usuarios con capacidad de logros mediante invocación a método `decorarConLogros` que envuelve instancia de `Usuario` en `DecoradorLogros`, retornando referencia a `ComponenteUsuario` que código cliente manipula uniformemente sin distinguir decoración. El sistema verifica cumplimiento de condiciones de logros mediante observadores que monitorean eventos relevantes como victoria sin recibir punto, completar nivel en tiempo límite o alcanzar umbral de puntaje, invocando `agregarLogro` en decorador cuando detección ocurre. La arquitectura permite agregar decoradores adicionales en futuro como `DecoradorEstadisticas` que registre métricas detalladas de rendimiento o `DecoradorRedSocial` que integre compartir logros en plataformas externas, componiendo múltiples decoradores mediante encadenamiento recursivo como `DecoradorRedSocial` envolviendo `DecoradorEstadisticas` envolviendo `DecoradorLogros` envolviendo `Usuario`.

4.12. Patrón Observer

El patrón Observer fue implementado para gestionar notificación automática de eventos relevantes del juego a múltiples subsistemas interesados sin acoplamiento directo entre emisor y receptores. Esta elección responde a la necesidad de coordinar actualizaciones de interfaz gráfica, reproducción de efectos sonoros contextuales y verificación de logros cuando eventos como cambios de puntaje, terminación de partida, completar de nivel o generación de ítems ocurren, evitando que `ModeloJuego` mantenga referencias explícitas a subsistemas específicos que incrementarían acoplamiento dificultando evolución independiente.

La interfaz `ObservadorJuego` define contrato común estableciendo métodos de notificación correspondientes a eventos observables, incluyendo `alCambiarPuntaje` invocado cuando puntaje de jugador se incrementa, `alTerminarJuego` invocado cuando condición de victoria o derrota se satisface, `alCompletarNivel` invocado cuando todos los bloques destructibles han sido eliminados en modo Breakout, y `alGenerarItem` invocado cuando ítem temporal es creado. Esta abstracción permite que observadores concretos implementen respuestas especializadas a eventos relevantes ignorando notificaciones de eventos que no les conciernen.

La clase `ObservadorUI` implementa observador especializado en actualizaciones de interfaz gráfica manteniendo referencia a `VistaJuego` que gestiona componentes visuales. El método `alCambiarPuntaje` invoca `actualizarPuntaje` en vista proporcionando valor actualizado que se renderiza en etiqueta correspondiente. El método `alTerminarJuego` invoca `mostrarDialogoFinJuego` en vista que presenta panel modal con información de resultado incluyendo victoria o derrota, puntaje final y estadísticas de partida. El método `alCompletarNivel` invoca `mostrarMensajeCompletado` que presenta notificación temporal indicando progreso. El método `alGenerarItem` invoca `mostrarAnimacionItem` que renderiza efecto visual transitorio indicando ubicación de aparición de ítem.

La clase `ObservadorSonido` implementa observador especializado en reproducción de audio manteniendo referencia a `reproductorSonido` que gestiona efectos sonoros. El método `alCambiarPuntaje` invoca `reproducirEfecto` especificando identificador de sonido de incremento de puntaje. El método `alTerminarJuego` invoca `reproducirMusicaFinJuego` que cambia pista de audio de fondo según resultado de partida. El método `alCompletarNivel` invoca `reproducirFanfarria` que ejecuta secuencia sonora de celebración. El método `alGenerarItem` invoca `reproducirEfectoItem` que emite tono indicando aparición de power-up.

La clase `ObservadorLogros` implementa verificación de condiciones de desbloqueo manteniendo referencia a `gestorLogros` que coordina reconocimientos. El método `alTerminarJuego` verifica si puntaje final cumple umbrales de logros configurados como obtener primera victoria, alcanzar racha de victorias consecutivas o completar modo en dificultad máxima sin perder punto, invocando desbloquear en `gestorLogros` cuando detección ocurre. Los métodos de otros eventos implementan verificaciones similares evaluando condiciones específicas asociadas a reconocimientos relevantes.

La clase `GestorObservadores` coordina registro y notificación manteniendo colección observadores que almacena referencias a `ObservadorJuego` registrados. El método `agregarObservador` inserta observador en

colección verificando que no haya sido previamente registrado. El método `eliminarObservador` remueve observador de colección. Los métodos de notificación como `notificarCambioPuntaje`, `notificarFinJuego`, `notificarCompletarNivel` y `notificarGenerarItem` iteran colección invocando método correspondiente en cada observador registrado, propagando evento a todos los interesados sin conocimiento de tipos concretos.

La integración del patrón en `ModeloJuego` se materializa mediante mantenimiento de referencia a `GestorObservadores`, invocando métodos de notificación apropiados cuando eventos relevantes ocurren durante actualización de estado. Por ejemplo, cuando pelota atraviesa línea defensiva incrementando puntaje, modelo invoca `notificarCambioPuntaje` en gestor que propaga evento a observadores registrados desencadenando actualización de interfaz, reproducción de sonido y verificación de logros simultáneamente. Esta arquitectura permite agregar observadores adicionales en futuro como `ObservadorAnalytics` que registre telemetría para análisis de comportamiento de jugadores o `ObservadorReplay` que grabe secuencia de estados para reproducción posterior, sin modificar `ModeloJuego` que permanece desacoplado de subsistemas específicos.

Descripción del Diagrama de Clases

El diagrama de clases del sistema documenta arquitectura completa mediante representación visual de todas las clases implementadas, sus atributos y métodos con visibilidades apropiadas, y relaciones estructurales incluyendo herencia, implementación de interfaces, asociaciones, composiciones y agregaciones. La organización del diagrama agrupa clases por responsabilidad funcional facilitando comprensión de subsistemas especializados y sus interacciones.

El núcleo arquitectónico se fundamenta en tres clases principales que implementan patrón MVC: `ModeloJuego` que encapsula estado completo del juego gestionando colección de objetos activos, gestores de colisiones e items, puntajes y referencias a gestores singleton, proporcionando métodos actualizar que procesa física y lógica, reiniciar que restaura configuración inicial, y operaciones para agregar y eliminar objetos dinámicamente. `VistaJuego` que gestiona interfaz gráfica con atributos lienzo de tipo `Canvas` donde renderiza elementos, contexto gráfico que proporciona primitivas de dibujo, etiquetas para visualización de puntajes, raíz de tipo `BorderPane` que organiza layout, y referencia a configuración singleton, proporcionando métodos renderizar que dibuja estado observable, `actualizarPuntaje` que modifica etiquetas, y `mostrarOcultarMenuPausa` que controla visibilidad de panel de opciones. `ControladorJuego` que coordina interacciones manteniendo referencias a modelo y vista, adaptador de entrada que captura eventos de teclado, y bucle de juego implementado mediante `AnimationTimer`, proporcionando métodos iniciar que activa bucle principal, pausar y reanudar que controlan ciclo de vida, y manejadores de eventos que procesan input delegando operaciones al modelo.

La jerarquía de objetos del juego implementa patrón Composite mediante clase abstracta `ObjetoJuego` que define interfaz común con atributos `x` e `y` representando posición, y métodos abstractos actualizar, dibujar, obtenerLimites, junto con getters para coordenadas. Las clases concretas `Pelota`, `Paleta` y `Bloque` extienden `ObjetoJuego` especializando comportamiento con atributos adicionales específicos: `Pelota` incluye `velocidadX`, `velocidadY`, `radio`, `rapidez` y `color`, con métodos `establecerVelocidad`, `incrementarRapidez` y `reiniciar`. `Paleta` incluye `ancho`, `alto`, `rapidez`, `color`, `tieneEspinass`, `estrategiaMovimiento` de tipo `EstrategiaMovimiento`, y `estadoOriginal`, con métodos `mover`, `establecerEstrategiaMovimiento`, `agregarEspinass`, `eliminarEspinass`, `redimensionar`, `clonar`, `guardarEstado` y `restaurarEstado`. `Bloque` incluye `ancho`, `alto`, `destruible`, `salud`, `color` y `puntos`, con métodos `golpear` que decrementa salud, `estaDestruido` que verifica eliminación, y `obtenerPuntos`. La clase `ObjetoJuegoCompuesto` implementa nodo compuesto con atributo hijos de tipo `List` que almacena `ComponenteJuego` subordinados, proporcionando métodos `agregar`, `eliminar`, `obtenerHijos`, y `sobreescrituras` de `actualizar` y `dibujar` que procesan recursivamente colección.

El sistema de colisiones implementa patrón Strategy mediante interfaz `EstrategiaColision` que define métodos `manejarColision` y `verificarColision`, con implementaciones concretas `EstrategiaColisionPelotaPaleta` que incluye método privado `calcularAnguloRebote` para rebotes controlados, `EstrategiaColisionPelotaBloque` que incluye método privado `generarItem` para creación probabilística de power-ups, y `EstrategiaColisionPelotaPared` que implementa rebotes simples. La clase `GestorColisiones` actúa como contexto manteniendo atributo estrategias de tipo `Map` asociando claves con instancias de `Estrategia-`

Colision, proporcionando métodos registrarEstrategia, verificarTodasColisiones y manejarColision que delegan procesamiento a estrategia apropiada.

El sistema de movimiento e inteligencia artificial implementa patrón Strategy paralelamente mediante interfaz EstrategiaMovimiento que define método calcularMovimiento retornando enumeración Direccion con valores ARRIBA, ABAJO y NINGUNA. Las implementaciones concretas incluyen EstrategiaMovimientoJugador que mantiene adaptadorEntrada para consultar estado de teclado, y EstrategiaMovimientoIAMedio que incluye atributo factorPrediccion configurando agresividad de anticipación. La enumeración DificultadIA define niveles FACIL, MEDIO y DIFICIL. La clase FabricaIA proporciona método público crearIA y métodos privados crearIAFacil, crearIAMedio y crearIADifcil que instancian estrategias especializadas. La clase ServicioIA coordina sistema manteniendo fabricaIA y dificultadActual, proporcionando métodos establecerDificultad, obtenerEstrategiaMovimiento y calcularSiguienteMovimiento.

El sistema de niveles implementa patrón Factory mediante clase Nivel que encapsula atributos id, nombre, dificultad, bloques de tipo List, tiempoLimite, y bloqueados, proporcionando métodos agregarBloque, eliminarBloque, estaCompletado que verifica condición de victoria, y cargar con guardar para persistencia. La clase FabricaMapasPredefinidos proporciona método crearNivel y métodos privados crearNivelClasico, crearNivelIntermedio y crearNivelDifcil que generan configuraciones optimizadas. La clase ServicioNiveles coordina acceso manteniendo fabricaNiveles y nivelesPersonalizados, proporcionando métodos cargarNivel que consulta predefinidos o personalizados, y guardarNivelPersonalizado que persiste creaciones de usuario.

El sistema de modos de juego implementa patrón Template Method mediante clase abstracta ModoJuegoAbstracto que mantiene atributos modeloJuego, controladorJuego y gestorPuntajes, definiendo método template jugar que invoca métodos gancho inicializarJuego, actualizarJuego, verificarCondicionVictoria, verificarCondicionDerrota y finalizarJuego que subclases especializan. Las implementaciones concretas ModoClasico que incluye atributo puntajeMaximo y ModoBreakout que incluye atributo vidasRestantes proporcionan semánticas especializadas implementando métodos gancho según reglas de modalidad.

El sistema de items temporales utiliza interfaz Item que define métodos aplicar, obtenerDuracion, estaActivo y desactivar. Las implementaciones concretas incluyen ItemVelocidad que mantiene incrementoVelocidad, duracion y activo con métodos privados guardarEstadoOriginal y restaurarEstadoOriginal, ItemMultiBola que mantiene cantidadBolas, duracion y activo, ItemRedimensionarPaleta que incluye multiplicadorTamano, duracion, activo y métodos privados de gestión de estado, y ItemEspinas que mantiene cantidadEspinas, duracion, activo y métodos privados similares. La clase GestorItems coordina ciclo de vida manteniendo itemsActivos y generadorAleatorio, proporcionando métodos generarItem que aplica probabilidad y selecciona tipo, actualizarItems que procesa expiración, y limpiarItems que remueve todos los activos.

El sistema de construcción de niveles implementa patrón Builder mediante clase ConstructorMapa que mantiene nivel y bloques, proporcionando métodos fluidos reiniciar, establecerNombre, establecerDificultad, agregarBloque, agregarBloqueDestructible, agregarBloqueIndestructible, agregarPatronBloques y método final construir que retorna Nivel validado. La enumeración TipoBloque define categorías DESTRUCTIBLE, INDESTRUCTIBLE, BONUS y MULTI_GOLPE.

El sistema de personalización de paletas implementa patrón Prototype mediante clase PrototipoPaleta que mantiene prototipos de tipo Map asociando nombres con instancias de Paleta, proporcionando métodos registrarPrototipo, clonarPaleta y crearPaletaPersonalizada que acepta ConfigPaleta. La clase ConfigPaleta encapsula ancho, alto, rapidez, colorPrimario y colorSecundario, proporcionando método aplicarA que modifica Paleta. La clase Paleta implementa método clonar que retorna copia profunda.

El sistema de gestores singleton incluye tres clases: GestorPuntajes que mantiene atributo estático privado instancia, constructor privado, atributos puntajesAltos, sesionActual y servicioPersistencia, proporcionando método estático obtenerInstancia, agregarPuntaje, obtenerPuntajesAltos, guardarPuntajes, cargarPuntajes y actualizarSesionActual. ConfiguracionJuego que mantiene instancia estática, constructor privado, atributos anchoPantalla, altoPantalla, rapidezPelota, rapidezPaleta y volumen,

proporcionando `obtenerInstancia`, `cargarConfiguracion`, `guardarConfiguracion`, `establecerVolumen` y getters. `GestorPrototiposPaleta` que mantiene instancia estática, constructor privado, atributo prototipos de tipo `PrototipoPaleta`, proporcionando `obtenerInstancia`, `registrarPrototiposDefecto`, `obtenerPrototipo`, `guardarPrototipoUsuario` y `listarPrototiposDisponibles`. Las clases auxiliares incluyen `Puntaje` con atributos `nombreJugador`, `puntos`, `fecha` y `modoJuego`, y `PuntajeSesion` que extiende o compone `Puntaje`.

El sistema de usuarios implementa patrón Decorator mediante interfaz `ComponenteUsuario` que define `obtenerDescripcion`, `obtenerNivel` y `obtenerExperiencia`. La clase `Usuario` implementa `ComponenteUsuario` con atributos `id`, `nombreUsuario`, `clave`, `correo`, `nivel`, `experiencia`, `fechaCreacion` y `configPaletaPreferida`, proporcionando métodos `agregarExperiencia`, `guardarConfiguracionPaleta` y `obtenerConfiguracionPaleta`. La clase abstracta `DecoradorUsuario` implementa `ComponenteUsuario` manteniendo `usuarioDecorado`, delegando métodos al componente wrapedo. La clase `DecoradorLogros` extiende `decorador` con atributo `logros` de tipo `List`, proporcionando métodos `agregarLogro`, `obtenerLogros` y sobreescritura de `obtenerDescripcion`. La clase `Logro` encapsula `id`, `nombre`, `descripcion`, `icono` y `desbloqueadoEn`, proporcionando métodos `desbloquear` y `estaDesbloqueado`.

El sistema de observadores implementa patrón Observer mediante interfaz `ObservadorJuego` que define métodos `alCambiarPuntaje`, `alTerminarJuego`, `alCompletarNivel` y `alGenerarItem`. Las implementaciones concretas incluyen `ObservadorUI` que mantiene `vistaJuego`, `ObservadorSonido` que mantiene `reproductorSonido`, y `ObservadorLogros` que mantiene `gestorLogros`, cada una implementando respuestas especializadas a eventos relevantes. La clase `GestorObservadores` coordina notificaciones manteniendo observadores de tipo `List`, proporcionando `agregarObservador`, `eliminarObservador`, `notificarCambioPuntaje`, `notificarFinJuego`, `notificarCompletarNivel` y `notificarGenerarItem` que iteran colección propagando eventos.

El sistema de autenticación incluye clase `ServicioAutenticacion` que mantiene usuarios de tipo `Map` asociando nombres con instancias de `Usuario`, `usuarioActual` y `codificadorClave`, proporcionando métodos `registrar`, `iniciarSesion`, `cerrarSesion`, `obtenerUsuarioActual`, `estaAutenticado`, `cambiarClave` y `decorarConLogros` que envuelve usuario en `DecoradorLogros`. La clase auxiliar `CodificadorClave` proporciona hashing seguro de contraseñas.

El sistema de persistencia incluye clase `ServicioPersistencia` que mantiene `manejadorArchivos`, proporcionando métodos `guardarUsuario`, `cargarUsuario`, `guardarNivel` y `cargarNivel` que serializan objetos en formato JSON mediante `Gson`. La clase auxiliar `ManejadorArchivos` gestiona operaciones de entrada-salida.

El sistema de entrada incluye clase `AdaptadorEntrada` que captura eventos de teclado manteniendo mapa de teclas presionadas, utilizado por `ControladorJuego` y `EstrategiaMovimientoJugador` para procesar input del usuario.

Las relaciones estructurales incluyen herencia donde clases concretas extienden abstracciones como `ObjetoJuego`, implementación de interfaces donde clases concretas realizan contratos definidos por `EstrategiaColision`, `EstrategiaMovimiento`, `Item`, `ObservadorJuego` y `ComponenteUsuario`, composición donde clases contienen componentes que no existen independientemente como `ModeloJuego` posee `GestorColisiones` y `GestorItems`, y agregación donde clases mantienen referencias a componentes independientes como `Paleta` referencia `EstrategiaMovimiento` sin poseerla. La multiplicidad de asociaciones indica cardinalidad como `ModeloJuego` relacionado con múltiples `ObjetoJuego` mediante composición uno a muchos, y `GestorObservadores` relacionado con múltiples `ObservadorJuego` mediante agregación uno a muchos.

La organización del diagrama facilita identificación de patrones mediante análisis de estructuras características: MVC identificable por `ModeloJuego`, `VistaJuego` y `ControladorJuego` con relaciones de dependencia unidireccional. Composite identificable por jerarquía `ObjetoJuego` con `ObjetoJuegoCompuesto` conteniendo colección de `ComponenteJuego`. Strategy identificable por interfaces `EstrategiaColision` y `EstrategiaMovimiento` con múltiples implementaciones concretas y contextos que mantienen referencias a estrategias. Factory identificable por `FabricaIA` y `FabricaMapasPredefinidos` con métodos de creación retornando productos abstractos. Template Method identificable por `ModoJuegoAbstracto` con

método jugar invocando métodos gancho implementados en subclases concretas. Builder identificable por ConstructorMapa con métodos fluidos retornando this y método construir retornando producto. Prototype identificable por PrototipoPaleta con método clonarPaleta y Paleta con método clonar. Singleton identificable por clases con constructor privado, instancia estática y método obtenerInstancia. Decorator identificable por ComponenteUsuario con DecoradorUsuario conteniendo componente decorado y DecoradorLogros extendiendo decorador base. Observer identificable por ObservadorJuego con implementaciones concretas y GestorObservadores coordinando notificaciones.

Diagramas de Casos de Uso y Estados

El sistema incluye documentación mediante diagramas UML que complementan diagrama de clases proporcionando perspectivas adicionales sobre funcionalidad observable por usuarios y transiciones de estado que gobiernan comportamiento de items temporales.

6.1. Diagrama de Casos de Uso

El diagrama de casos de uso documenta funcionalidades principales del sistema desde perspectiva del actor Usuario, organizando capacidades en casos de uso que representan objetivos alcanzables mediante interacción con aplicación. Los casos de uso fundamentales incluyen Jugar Modo Clasico que permite participar en partida tradicional uno contra uno donde victoria se alcanza acumulando puntaje máximo configurado, Jugar Modo Breakout que permite destruir formaciones de bloques controlando paleta inferior evitando que pelota caiga, Editar Nivel Personalizado que permite diseñar mapas mediante colocación arbitraria de bloques con editor visual, Configurar Dificultad IA que permite seleccionar nivel de desafío de oponente artificial, Personalizar Paleta que permite modificar dimensiones, velocidad y colores de paleta propia, Registrar Cuenta que permite crear usuario nuevo con credenciales únicas, Iniciar Sesión que permite autenticarse proporcionando nombre y contraseña, Ver Puntajes Altos que permite consultar tabla de posiciones con mejores puntuaciones históricas, Ver Logros que permite visualizar reconocimientos desbloqueados con fechas de obtención, Guardar Progreso que permite persistir configuración actual y estado de partida para reanudación posterior, y Cargar Partida que permite restaurar sesión previamente guardada.

Las relaciones entre casos de uso incluyen extensiones que indican funcionalidad opcional ejecutada condicionalmente, por ejemplo Generar Item extiende Jugar Modo Breakout indicando que creación de power-ups ocurre probabilísticamente durante destrucción de bloques pero no constituye paso obligatorio del flujo principal. Las inclusiones indican subfuncionalidades requeridas obligatoriamente, por ejemplo Jugar Modo Clasico incluye Configurar Dificultad IA dado que modo requiere oponente artificial cuyo comportamiento debe ser especificado antes de iniciar partida. Las generalizaciones agrupan casos de uso especializados bajo abstracción común, por ejemplo Jugar Modo Clasico y Jugar Modo Breakout generalizan a caso de uso abstracto Jugar Partida que encapsula flujo común de inicialización, bucle principal y finalización compartido por todas las modalidades.

El actor Usuario interactúa con sistema mediante invocación de casos de uso representados mediante óvalos etiquetados con nombres descriptivos. Las líneas de asociación conectan actor con casos de uso indicando capacidad de invocar funcionalidad. Las relaciones de extensión utilizan flecha punteada con estereotipo extend apuntando desde caso de uso extendido hacia caso base. Las relaciones de inclusión utilizan flecha punteada con estereotipo include apuntando desde caso base hacia caso incluido. Las relaciones de generalización utilizan flecha sólida con punta vacía apuntando desde caso especializado hacia caso general.

El diagrama proporciona visión de alto nivel de qué puede hacer el sistema desde perspectiva del usuario final, facilitando validación de que requisitos funcionales especificados están completamente cubiertos por casos de uso implementados. La organización clara de funcionalidades facilita comunicación con stakeholders no técnicos que pueden comprender capacidades del sistema sin conocimiento de arquitectura interna o detalles de implementación.

6.2. Diagrama de Estados

El diagrama de estados documenta ciclo de vida de items temporales que modifican comportamiento de objetos del juego mediante transiciones entre estados que gobiernan aplicación, duración y desactivación de efectos. El diagrama representa estados como rectángulos redondeados etiquetados con nombres descriptivos, transiciones como flechas dirigidas conectando estados con etiquetas indicando eventos que desencadenan cambio, y acciones como texto asociado a transiciones o estados indicando operaciones ejecutadas durante transición o permanencia.

El ciclo inicia en estado Inactivo que representa item no existente antes de generación. La transición Generar desencadenada por destrucción de bloque con probabilidad configurada provoca cambio a estado Generado ejecutando acción de instanciación de objeto Item con tipo seleccionado aleatoriamente de catálogo disponible y posición establecida a ubicación de bloque destruido. El estado Generado representa item visible en campo esperando recolección por paleta o pelota. La transición Recolectar desencadenada por colisión entre item y objeto recolector provoca cambio a estado Activo ejecutando acción aplicar que invoca transformación sobre objeto host modificando propiedades según tipo de efecto. El estado Activo representa item aplicado cuyo efecto está vigente durante duración configurada. La permanencia en estado Activo ejecuta acción de actualización que decrementa contador de duración en cada frame del bucle de juego. La transición Expirar desencadenada automáticamente cuando duración alcanza cero provoca cambio a estado Desactivado ejecutando acción desactivar que invoca reversión de transformación restaurando propiedades originales de objeto host mediante consulta de estado guardado previamente. El estado Desactivado representa item que completó ciclo de vida y será removido de colección de items activos en próxima actualización del gestor. La transición Limpiar desencadenada por actualización de GestorItems provoca cambio al estado final representado por círculo con borde doble, eliminando item de memoria liberando recursos.

El diagrama incluye estado especial Permanente accesible mediante transición desde Activo para items cuyo efecto no expira automáticamente como múltiples pelotas que persisten hasta ser eliminadas mediante condiciones normales de juego. La transición a Permanente omite acción de desactivación automática requiriendo limpieza explícita durante finalización de partida o reinicio de nivel.

Las anotaciones en el diagrama especifican condiciones de guarda entre corchetes que restringen aplicabilidad de transiciones, por ejemplo transición Recolectar anotada con guarda que verifica que objeto recolector no tiene item del mismo tipo activo previniendo acumulación redundante que podría causar efectos desbalanceados. Las acciones de entrada y salida especificadas como entry y exit indican operaciones ejecutadas al ingresar o abandonar estado independientemente de transición específica utilizada.

El diagrama proporciona comprensión clara de comportamiento temporal de items facilitando implementación correcta de lógica de gestión de estados, validación de que todas las transiciones posibles están contempladas evitando estados inconsistentes, y comunicación de semántica de items a diseñadores de jugabilidad que ajustan duraciones y efectos para balance óptimo.

Finalmente

El proyecto demuestra aplicación práctica exhaustiva de doce patrones de diseño que colaboran coordinadamente para resolver problemática compleja de desarrollo de videojuego moderno con funcionalidades extendidas, cumpliendo requisitos funcionales establecidos mediante arquitectura bien estructurada que adhiere a principios SOLID maximizando mantenibilidad, extensibilidad y testabilidad del sistema desarrollado. La separación estricta de responsabilidades mediante patrón MVC proporciona base arquitectónica sobre la cual patrones adicionales resuelven desafíos especializados de composición jerárquica, algoritmos intercambiables, creación de objetos complejos, comportamiento dinámico, extensión transparente y notificación desacoplada, resultando en sistema profesional capaz de evolucionar sosteniblemente sin acumular deuda técnica que comprometa viabilidad a largo plazo.

La implementación de patrones Composite, Strategy, Factory, Template Method, Builder, Prototype, Singleton, Decorator y Observer permite que sistema soporte extensiones futuras como nuevos modos

de juego, items temporales adicionales, niveles de dificultad de inteligencia artificial personalizados, integración con redes sociales para compartir logros, modo multijugador en línea mediante networking, y personalización visual avanzada con shaders programables, sin requerir refactorizaciones arquitectónicas mayores que desestabilizarían funcionalidad existente. La centralización de responsabilidades en clases especializadas facilita localización y corrección de defectos mediante aislamiento de componentes afectados, mientras pruebas unitarias permiten validación automatizada de comportamiento esperado detectando regresiones introducidas durante modificaciones.

La revitalización del concepto clásico de Pong mediante incorporación de funcionalidades modernas demuestra que diseño arquitectónico sólido fundamentado en patrones probados permite transformar idea simple en producto competitivo capaz de atraer audiencia contemporánea exigente, preservando accesibilidad inmediata que caracteriza al diseño original mientras proporciona profundidad suficiente para mantener engagement prolongado mediante variedad de modos, progresión cuantificable y desafíos adaptativos. El sistema desarrollado constituye base extensible para evolución continua incorporando retroalimentación de usuarios, tendencias emergentes de diseño de videojuegos, y tecnologías innovadoras que enriquezcan experiencia sin comprometer estabilidad de funcionalidad central.